

Paradigma de Linguagem de Programação – PLP

Exercício Aula 8

Para cada trecho: (a) qual é a saída? (b) qual conceito da aula explica?

1 · Python

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

Console output: [1], [1, 2]

a) [1] [1, 2]

b) Conceito: Argumento padrão mutável. A lista é criada uma única vez e reutilizada nas próximas chamadas, dessa forma 2 é adicionado na mesma lista que o 1 estava

2 · Java

```
import java.util.*;  
  
public class Main {  
    static void zera(int[] v, int n) {  
        v[0] = 0;  
        n = 0;  
    }  
  
    public static void main(String[] args) {  
        int[] v = {5, 5};  
        int n = 5;  
  
        zera(v, n);  
  
        System.out.println(v[0] + " " + n);  
    }  
}
```

Console output: 0 5

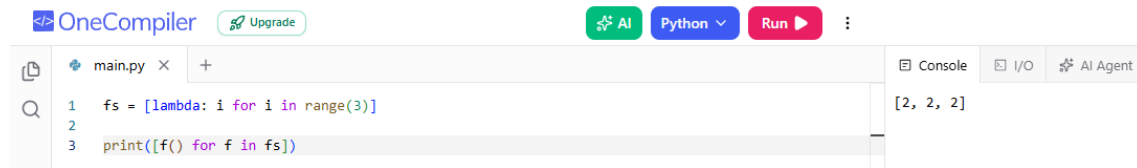
a) 0 5

b) Conceito: Passagem de parâmetros em JAVA.

(v) é um array e ao passar o array para (zera), a função consegue alterar seu conteúdo, então v[0] passa de 5 para 0.

(n) é um int (tipo primitivo). Java passa esse valor por cópia, então n = 0 dentro de (zera) não altera o (n) original.

3 · Python



The screenshot shows the OneCompiler Python IDE. The code in the editor is:

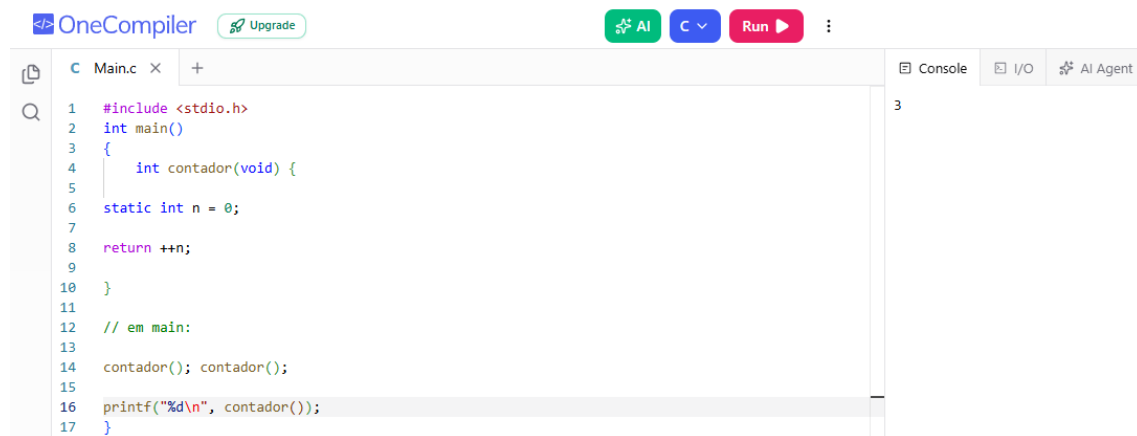
```
1 fs = [lambda: i for i in range(3)]
2
3 print([f() for f in fs])
```

The console output on the right is `[2, 2, 2]`.

a) [2, 2, 2]

b) Conceito: Closure ou late binding (Ligação tardia). As (lambdas) não guardam o valor de (i) no mesmo momento que elas são criadas, mas sim uma referência da var(i), por isso quando as funções são executadas, o for já terminou e (i) vale 2

4 · C



The screenshot shows the OneCompiler C IDE. The code in the editor is:

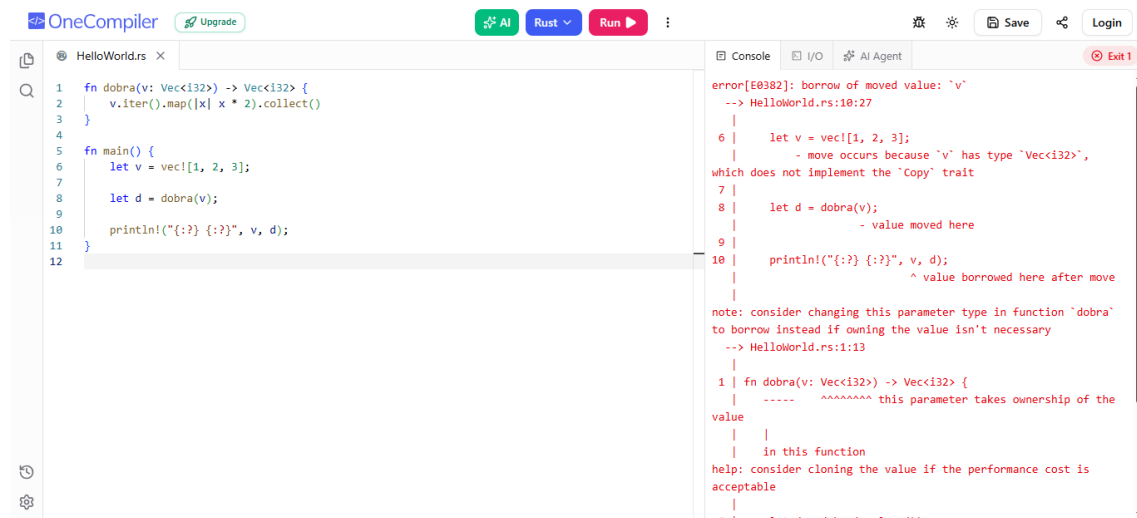
```
1 #include <stdio.h>
2 int main()
3 {
4     int contador(void) {
5
6     static int n = 0;
7
8     return ++n;
9
10    }
11
12    // em main:
13
14    contador(); contador();
15
16    printf("%d\n", contador());
17 }
```

The console output on the right is `3`.

a) 3

b) Conceito: variável Static local. (n) não é recriado a cada chamada. Ele apenas mantém seu valor entre as chamadas.

5 · Rust



The screenshot shows the OneCompiler Rust IDE. The code in the editor is:

```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7
8     let d = dobra(v);
9
10    println!("{:?} {:?}", v, d);
11 }
12
```

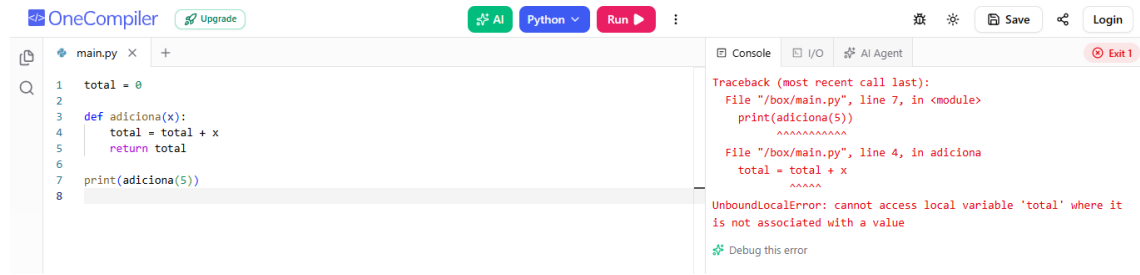
The console output on the right shows an error:

```
error[E0382]: borrow of moved value: `v`
--> HelloWorld.rs:10:27
   |
6  |     let v = vec![1, 2, 3];
   |     - move occurs because `v` has type `Vec<i32>`,
   |     which does not implement the `Copy` trait
7  |
8  |     let d = dobra(v);
   |               - value moved here
9  |
10 |     println!("{:?} {:?}", v, d);
   |                          ^ value borrowed here after move
   |
note: consider changing this parameter type in function `dobra`
to borrow instead of owning the value isn't necessary
--> HelloWorld.rs:1:13
   |
1  | fn dobra(v: Vec<i32>) -> Vec<i32> {
   |     ^^^^^ this parameter takes ownership of the value
   |
   | in this function
help: consider cloning the value if the performance cost is acceptable
   |
9  |     let d = dobra(v.clone());
```

a) Código não compila.

b) Conceito: Ownership ou move to Rust. No Rust, em nível global você não pode simplesmente executar let dessa forma. Para ver o código rodando corretamente, altere a função para receber uma referência `let d = dobra(&v)`;

6 · Python



The screenshot shows the OneCompiler Python IDE. The code in `main.py` is as follows:

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
8
```

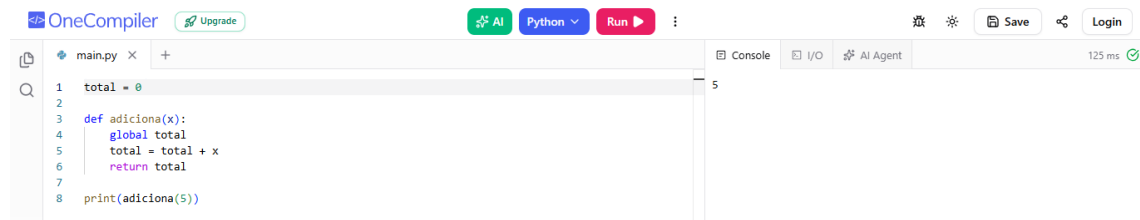
The console displays a `Traceback` error:

```
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ~~~~~
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ~~~~~
UnboundLocalError: cannot access local variable 'total' where it
is not associated with a value
```

Buttons for AI, Python, Run, Save, and Login are visible at the top.

a) Não imprime corretamente

b) Conceito: escopo de variáveis. o Python entende que `total` é uma variável local da função, podemos corrigir alterando a var (`total`) para global:



The screenshot shows the OneCompiler Python IDE with the corrected code in `main.py`:

```
1 total = 0
2
3 def adiciona(x):
4     global total
5     total = total + x
6     return total
7
8 print(adiciona(5))
```

The console shows the output `5` and a status bar indicating `125 ms` execution time.